

Обновление и сопровождение REST API в Production

Release workflow, migrations, restart/redeploy, health checks, smoke tests, logs и rollback

Цели лекции

- Понять жизненный цикл production-версии REST API.
- Разобрать безопасный порядок обновления Node.js REST API на Linux-сервере.
- Научиться проверять новую версию через health checks и smoke tests.
- Разобрать обновление на PaaS Render и различия с VPS/Linux.
- Понять роль database backup перед изменением схемы.
- Разобрать rollback приложения и ограничения rollback базы данных.
- Сформировать production checklist сопровождения REST API.

Связь с предыдущими лекциями

- Лекция 5 подготовила приложение к production: config, health checks, graceful shutdown.
- Лекция 6 добавила CI: автоматический запуск Jest/Supertest.
- Лекция 7 показала deployment на Render.
- Лекция 8 показала manual deployment на Linux через systemd и Nginx.
- Лекция 9 отвечает на вопрос: что делать после того, как API уже опубликован.



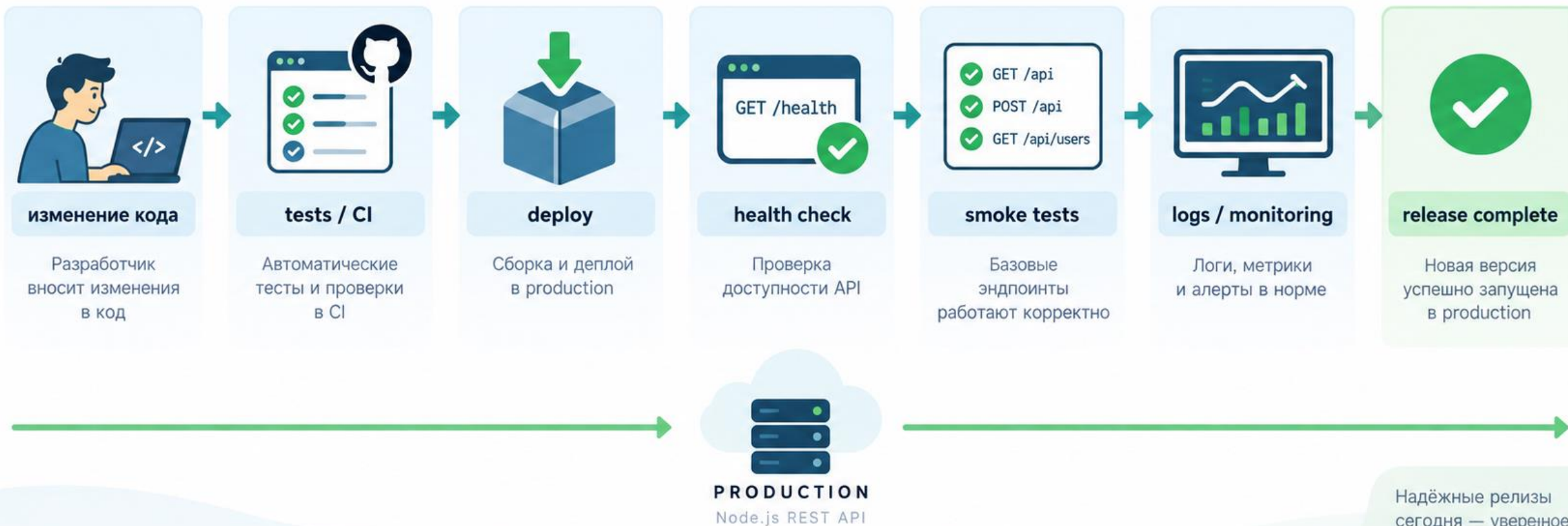
1. Жизненный цикл production-версии REST API

От изменения кода до подтверждённого release

Стабильные
сервисы.
Больше возможностей.

Production release lifecycle

Node.js REST API — от изменения кода до стабильной версии в продакшене



DEPLOY > MONITOR > DELIVER VALUE

Надёжные релизы
сегодня — уверенное
развитие завтра.

Что такое release

- Release — опубликованная версия приложения, которая прошла проверку и стала доступна пользователям или внешним клиентам API.
- Release связан с конкретным commit или tag.
- Release должен быть воспроизводимым.
- После release нужно проверить реальную работоспособность API.
- Если новая версия сломала production, нужен понятный rollback plan.

Deployment и Release

Deployment

- Техническое размещение новой версии.
- Получить код.
- Установить dependencies.
- Применить migrations.
- Перезапустить service.

Release

- Подтверждение, что версия реально работает.
- Health checks проходят.
- Smoke tests проходят.
- Logs не показывают новых ошибок.
- Команда фиксирует успешный выпуск.

Минимальный production workflow

- 1 Prepare** Проверить изменения, tests и readiness к deployment.
- 2 Backup** Сделать backup, если deployment затрагивает database.
- 3 Deploy** Получить новую версию и подготовить dependencies.
- 4 Migrate** Применить database migrations.
- 5 Restart** Перезапустить Node.js service или дождаться redeploy платформы.
- 6 Verify** Проверить health, smoke tests и logs.

Что может пойти не так

- Новая версия не запускается из-за отсутствующей `environment variable`.
- `npm ci` падает из-за несоответствия `package.json` и `package-lock.json`.
- Migration не применяется или несовместима с текущими данными.
- API запускается, но `protected endpoints` возвращают неожиданные 401/403/500.
- Frontend или внешний клиент получает CORS error.
- Logs показывают рост ошибок после deployment.

Стабильные релизы — результат дисциплины, а не удачи.

Контрольные точки release

Node.js REST API в продакшене — это последовательность проверок, а не одно действие.



Надёжный API. Больше возможностей.

До release

Готовим и проверяем всё необходимое

1



Код готов

Feature/Fix влит в main, выбрана версия релиза

2



CI PASS

Пройдены unit и интеграционные тесты

3



Config проверен

Проверены production переменные окружения и секреты

4



Миграции готовы

Prisma/PostgreSQL миграции проверены, подготовлен план backup и rollback

5



Deploy

Новая версия развернута в production

6



Health / Smoke

Пройден health check и ключевые smoke-тесты API

7



Monitoring

Логи, метрики и уровень ошибок в норме. Релиз принят



Stop / Rollback / Fix

Если на любом шаге проверка не пройдена

НАДЁЖНЫЕ РЕЛИЗЫ
ДЕЛАЮТ ПРОДУКТ СИЛЬНЕЕ

Проверяй. Выпускай.
Развивай дальше.



2. Подготовка к обновлению

Что проверить до изменения production

Pre-deploy checklist



Изменения закоммичены и находятся в Git.



Известен commit или tag, который будет развёрнут.



CI завершился success.



npm test проходит локально или в CI.



Есть список изменённых environment variables.



Известно, есть ли database migration.



Есть план rollback.



Есть доступ к logs production-сервиса.

Почему нельзя обновлять production наугад

- Без commit/tag сложно понять, какая версия сейчас работает.
- Без CI можно развернуть код с падающими tests.
- Без backup опасно менять database schema.
- Без smoke tests можно не заметить сломанную авторизацию или ключевой endpoint.
- Без logs невозможно быстро понять причину ошибки.

Проверка Git перед release

BASH

```
git status  
git log --oneline -5  
git rev-parse --short HEAD
```

Release marker

- Release marker — конкретный идентификатор версии, который помогает понять, что именно запущено в production.
- Commit hash.
- Git tag.
- Версия в package.json.
- Отдельный /version endpoint.
- Запись в release notes.

Простой /version endpoint

JAVASCRIPT

```
app.get('/version', (req, res) => {  
  res.json({  
    name: 'internet-shop-api',  
    version: process.env.APP_VERSION ?? 'dev'  
  });  
});
```

Что важно

- Endpoint помогает понять, какая версия приложения отвечает.
- APP_VERSION можно задавать в systemd EnvironmentFile или Render Environment.
- Не возвращайте secrets, DATABASE_URL или подробности инфраструктуры.

3. Обновление REST API на Linux-сервере

Git pull, npm ci, Prisma, systemd и Nginx

Plan
Update
Verify
Stay safe ✓

Linux update flow

A safe and structured approach to keep your Linux server (Node.js REST API) up to date



Stable server
Reliable API
Happy users

1

Check maintenance window / backup



Choose a low-traffic time window



Take a backup / snapshot
(disk, DB, config)

2

Refresh package lists

```
$ sudo apt update
```



Download the latest package information from repositories.

3

Review available updates

```
$ apt list --upgradable  
  
Listing...  
nodejs/now 20.11.1 → 20.11.2  
nginx/now 1.24.0 → 1.24.1  
postgresql/now 16.1 → 16.2  
...  
5 upgraded, 0 newly installed
```



Check what will be updated (e.g. security updates).

4

Apply updates

```
$ sudo apt upgrade -y
```



Install the available updates safely (non-interactive).

5

Restart services or reboot if required

```
$ sudo systemctl restart <service>  
# or reboot  
$ sudo reboot
```



Restart updated services (e.g. nginx, node app) or reboot if the kernel was updated.

6

Verify application health

- ✓ Node.js apt responding
- ✓ Nginx working (reverse proxy)
- ✓ PostgreSQL accessible
- ✓ Health endpoint OK

```
$ curl -f http://localhost:3000/health  
OK
```

Run smoke tests and confirm the API and dependencies are healthy.

7

Monitor logs

```
$ journalctl -u <service>  
$ tail -f /var/log/nginx/  
$ tail -f /var/log/app.log
```



Watch logs for errors or unusual behavior after the update.

✓

Server updated and healthy



Your Linux server and Node.js REST API are up to date and running smoothly.



Tip: Test this process in a staging environment first, and keep backups so you can quickly recover if needed.



If problem detected → rollback / investigate

- Revert the problematic package (e.g. `apt install <package>=<version>`)
- Restore from backup if needed
- Check logs and investigate the issue
- Do not proceed until the server is stable

Fix and retry

Исходное состояние перед обновлением

- Приложение уже развёрнуто в `/var/www/internet-shop`.
- `systemd` service называется `internet-shop`.
- Environment variables находятся в `/etc/internet-shop.env`.
- PostgreSQL работает локально.
- Nginx проксирует запросы в Express.
- Health endpoints уже настроены.

Подключение к серверу

BASH

```
ssh ubuntu@SERVER_IP
```

Проверка текущего состояния

BASH

```
sudo systemctl status internet-shop --no-pager  
curl http://127.0.0.1:3000/health/ready  
curl http://SERVER_IP/health/ready
```

Получение новой версии

BASH

```
sudo -iu shop  
cd /var/www/internet-shop  
git fetch origin  
git log --oneline --decorate -5  
git pull --ff-only origin main
```

Что важно

- Работаем от пользователя приложения shop.
- git fetch получает информацию о новых commits.
- git pull --ff-only обновляет ветку без создания merge commit на сервере.
- Если fast-forward невозможен, deployment останавливаем и разбираемся в Git history.

Обновление dependencies

BASH

```
npm ci
```

Что важно

- npm ci синхронизирует node_modules с package-lock.json.
- Не используем npm install для случайного обновления lockfile на production-сервере.
- Если npm ci падает, приложение не перезапускаем новой версией.

Prisma generate и migrations

BASH

```
set -a  
source /etc/internet-shop.env  
set +a
```

```
npx prisma generate  
npx prisma migrate deploy
```

Почему порядок важен

- Сначала получаем новую версию кода.
- Затем устанавливаем dependencies.
- Затем генерируем Prisma Client под актуальную schema.
- Затем применяем migrations к базе.
- Только после успешных steps перезапускаем application service.

Перезапуск Node.js service

BASH

```
exit  
sudo systemctl restart internet-shop  
sudo systemctl status internet-shop --no-pager
```

Проверка logs после restart

BASH

```
sudo journalctl -u internet-shop -n 100 --no-pager  
sudo journalctl -u internet-shop -f
```

Health checks после update

BASH

```
curl http://127.0.0.1:3000/health/live  
curl http://127.0.0.1:3000/health/ready  
curl http://SERVER_IP/health/ready
```

Порядок безопасного Linux update

Обновляем систему надёжно, без простоев и с возможностью отката

Стабильность
Безопасность
Контроль



Обновляй
осознанно!

1

Проверка
текущего состояния



Узнаём версию ОС,
ядра и доступные
обновления.

```
$ cat /etc/os-release
$ uname -r
$ uptime
$ df -h
$ free -h
$ sudo apt update
```

Убедитесь, что система
работает стабильно
и нет критических
ошибок.

2

Резервное
копирование



Делаем бэкап важных
данных и конфигураций
(БД, /etc, приложения).

```
$ sudo pg_dump -U user
dbname > db.sql
$ sudo tar -czf
/etc-backup.tar.gz /etc
$ sudo cp -r /var/www
/backup/
```

Проверьте, что бэкапы
созданы и доступны
для восстановления.

3

Планирование
и проверка



Выбираем время
обновления (минимальная
нагрузка), проверяем
список пакетов.

```
$ sudo apt update
$ sudo apt list --upgradable
```

Проверьте, нет ли
конфликтов или
критических изменений
в зависимостях.

4

Остановка
сервисов (при
необходимости)



Останавливаем
приложения, чтобы
избежать ошибок
во время обновления.

```
$ sudo systemctl stop
node-api.service
$ sudo systemctl stop
nginx
```

Останавливайте только
нужные сервисы.
Можно использовать
режим обслуживания.

5

Установка
обновлений



Обновляем пакеты
и при необходимости
ядро системы.

```
$ sudo apt upgrade -y
$ sudo apt full-upgrade -y
$ sudo apt autoremove -y
```

Используйте
full-upgrade для
крупных обновлений
(включая ядро).

6

Проверка
после обновления



Проверяем версию,
статус сервисов
и работоспособность
приложения.

```
$ uname -r
$ sudo systemctl status
node-api.service
$ sudo systemctl status
nginx
$ curl http://localhost:3000
$ curl -I https://your-domain
```

Убедитесь, что все
сервисы запущены
и приложение работает
корректно.

7

Тестирование
функциональности



Проверяем основные
функции приложения
и мониторинг.

```
$ npm test
$ curl https://your-domain
/api/health
$ sudo journalctl -u
node-api.service -n 50
```

Проверьте API,
логи, метрики и
внешнюю доступность.

8

Возврат к работе
и мониторинг



Если всё в порядке —
система в работе.
Продолжаем наблюдение
(логи, метрики, алерты).

```
$ sudo systemctl start
node-api.service
$ sudo systemctl start
nginx
```

В течение нескольких
часов следите за
логами и метриками.



Обновление выполнено успешно!

Система обновлена, сервисы работают, всё под контролем.



Всегда делайте
резервные копии



Читайте changelog
перед обновлением



Тестируйте
после обновления



Настройте
мониторинг

Стабильная система —
основа надёжного сервиса!



4. Database migrations и backup

Как не потерять данные при изменении схемы

Почему database update рискованнее обычного кода

- Код можно быстро вернуть на предыдущий commit.
- Изменённую database schema не всегда можно мгновенно откатить.
- Некоторые migrations меняют или удаляют данные.
- Rollback приложения может не работать со схемой, которая уже изменилась.
- Перед важными migrations нужен backup.

Backup PostgreSQL через pg_dump

BASH

```
sudo -u postgres pg_dump shop_db > /tmp/shop_db_$(date +%Y%m%d_%H%M%S).sql
```

Что важно

- pg_dump создаёт SQL backup базы данных.
- Файл backup нужно хранить безопасно и не публиковать.
- Для production backup должен быть регулярным и проверяемым.
- Перед лабораторной migration достаточно учебного ручного backup.

Проверка backup file

BASH

```
ls -lh /tmp/shop_db_*.sql  
head -n 5 /tmp/shop_db_YYYYMMDD_HHMMSS.sql
```

Правила безопасных migrations

- Не удалять column сразу, если старая версия кода ещё может её использовать.
- Сначала добавить новое поле, затем обновить код, затем удалить старое поле отдельным release.
- Проверять migration на test database.
- Не использовать destructively commands без понимания последствий.
- Понимать, совместима ли migration с rollback приложения.

Безопасный и опасный подход к изменению схемы

Безопаснее

- Добавить nullable field.
- Заполнить данные отдельным шагом.
- Обновить код.
- Проверить production.
- Удалить старое поле позже.

Опаснее

- Удалить column сразу.
- Переименовать поле без переходного периода.
- Менять тип данных без проверки.
- Запустить migration без backup.
- Не иметь rollback plan.

Prisma migrate deploy

- Команда применяет уже созданные migrations.
- Она не создаёт новые migration files.
- Она предназначена для staging/production deployment workflows.
- Если migration failed, нужно остановить release и анализировать причину.
- После migration важно проверить endpoints, которые используют изменённую схему.

Database-safe release

Безопасные изменения базы данных при обновлении приложения



Миграции без простоя.
Сначала совместимость,
потом изменения.



- ✓ Данные в безопасности
- ✓ Без простоя
- ✓ Возможен откат
- ✓ Проверенная схема

1

Анализ изменений

Планируем изменения схемы и проверяем совместимость существующих данных.



- ✓ Что меняем?
- ✓ Влияет ли на существующие данные?
- ✓ Нужна ли миграция?
- ✓ Нужен ли откат?

2

Подготовка миграций

Пишем безопасные миграции (расширяем схему, не ломаем существующую логику).

```
-- Добавляем новый столбец
ALTER TABLE users
ADD COLUMN is_active BOOLEAN
DEFAULT true;

-- Создаем индекс
CREATE INDEX idx_users_email
ON users(email);
```

- ✓ Только обратимо (reversible)
- ✓ Добавляем, а не удаляем
- ✓ Используем индексы
- ✓ Без блокировки таблиц

3

Развертывание миграций

Применяем миграции до обновления приложения.



```
npx prisma migrate deploy
# или
npm run db:migrate
```

- ✓ Проверяем успешность
- ✓ Контролируем время выполнения
- ✓ Следим за блокировками
- ✓ Делаем бэкап перед миграцией

4

Обновление приложения

Деплоим новую версию API, которая поддерживает новую схему.



```
sudo systemctl restart
node-api.service
# или (Docker)
docker compose up -d
```

- ✓ Нулевая простоя
- ✓ Плавное обновление (blue/green или rolling)
- ✓ Проверяем healthcheck
- ✓ Совместимость со старой и новой схемой

5

Проверка и валидация

Убеждаемся, что приложение корректно работает с новыми данными.



- ✓ Проверяем API endpoints
- ✓ Тестируем ключевые сценарии
- ✓ Проверяем данные в базе
- ✓ Мониторим логи и метрики
- ✓ Проверяем производительность

6

Откат (при необходимости)

Если обнаружены проблемы, возвращаемся на предыдущую версию (миграции тоже обратимы).



```
npx prisma migrate resolve
--rolled-back <migration>
sudo systemctl restart
node-api.service
```

- ✓ Используем down-миграции
- ✓ Восстанавливаем бэкап (при необходимости)
- ✓ Возвращаем предыдущую версию приложения
- ✓ Проверяем работоспособность



Безопасные изменения = стабильный бизнес

Планируй. Тестируй. Развертывай. Проверяй. Будь готов к откату.



Резервные копии



Версионирование миграций



Мониторинг и алерты



План отката (down-миграции)

Меняем базу данных без риска!



5. Health checks и smoke tests

Как быстро доказать, что новая версия работает

Health check и smoke test

- Health check показывает базовую готовность сервиса, а smoke test проверяет ключевые пользовательские сценарии API.
- Health check обычно короткий и автоматизированный.
- Smoke tests выполняются после deployment.
- Они не заменяют весь test suite.
- Их цель — быстро обнаружить критическую поломку в production.

Минимальный набор проверок Internet Shop API

Проверка	Endpoint	Ожидаемый результат
Liveness	GET /health/live	200 OK
Readiness	GET /health/ready	200 OK и database доступна
Public catalog	GET /api/products	200 OK
Login	POST /api/auth/login	200 OK и token
Protected endpoint	GET /api/orders или admin endpoint	401/403/200 по ожидаемому сценарию
Version	GET /version	Новая версия или commit

Smoke test через curl

BASH

```
API=http://SERVER_IP
```

```
curl -i $API/health/live  
curl -i $API/health/ready  
curl -i $API/api/products  
curl -i $API/version
```

Smoke test login — cxema

BASH

```
curl -i -X POST $API/api/auth/login \  
  -H 'Content-Type: application/json' \  
  -d '{"email":"admin@example.com","password":"***"}'
```

Smoke test не должен портить production data

- GET-запросы безопаснее для базовой проверки.
- Если нужен POST/PATCH/DELETE, используйте тестовый объект и удаляйте его после проверки.
- Не используйте реальные пароли и токены в отчёте.
- Не создавайте мусорные production records без cleanup.
- Для регулярных проверок лучше иметь отдельные test accounts.

Post-deploy verification

Проверяем, что после релиза приложение работает корректно



- ✓ Приложение доступно
- ✓ Все ключевые функции работают
- ✓ Нет ошибок в логах
- ✓ Сервисы стабильны



Релиз успешен

Приложение работает корректно и готово к использованию

1

Доступность приложения

Проверяем, что сервис отвечает и возвращает корректный статус.



```
$ curl -I https://api.example.com
HTTP/2 200
...
```

- ✓ Статус 200 OK
- ✓ Нет ошибок подключения
- ✓ Корректный SSL-сертификат

2

Проверка API endpoints

Тестируем ключевые эндпоинты (CRUD, аутентификация и т.д.).

```
GET /api/items
POST /api/items
PUT /api/items/1
DELETE /api/items/1
```

```
$ curl -X GET \
https://api.example.com/api/items
[ { "id": 1, "name": "Item" } ]
```

- ✓ Основные эндпоинты возвращают корректные данные
- ✓ Проверяем ошибки (404, 400, 401)
- ✓ Работает авторизация (если требуется)

3

Проверка интеграций

Убедимся, что работают внешние сервисы (БД, кэш, очереди и др.).



```
$ curl https://api.example.com/api/health
{
  "status": "ok",
  "database": "ok",
  "redis": "ok",
  "mailer": "ok"
}
```

- ✓ Соединение с БД
- ✓ Работа Redis (кэш/sessions)
- ✓ Отправка писем (SMTP)
- ✓ Доступность других сервисов (API, очереди и т.д.)

4

Проверка логов

Смотрим логи приложения и nginx на наличие ошибок.



```
# Логи приложения
sudo journalctl -u node-api.service -n 100
# Логи nginx
sudo journalctl -u nginx -n 100
# В режиме реального времени
sudo journalctl -u node-api.service -f
```

- ✓ Нет критических ошибок
- ✓ Нет повторяющихся ошибок
- ✓ Проверяем предупреждения
- ✓ Логи соответствуют ожидаемому поведению

5

Проверка производительности

Контролируем время ответа и нагрузку на систему.



```
ab -n 1000 -c 10 \
https://api.example.com/
# или
npx autocannon -c 10 -d 30 \
https://api.example.com/
```

- ✓ Время ответа в пределах нормы
- ✓ Нет деградации производительности
- ✓ CPU/RAM/диски в норме
- ✓ Проверяем метрики (например, через Prometheus)

6

Проверка через Nginx

Убеждаемся, что прокси корректно перенаправляет запросы к приложению.



```
curl -I https://api.example.com
# проверяем заголовки
HTTP/2 200 OK
server: nginx
...
```

- ✓ HTTPS работает
- ✓ Корректные заголовки
- ✓ Перенаправление на приложение
- ✓ Нет ошибок 502/504

7

Функциональное тестирование

Проверяем основные сценарии работы приложения (вручную или автотестами).



```
# Запуск автотестов
npm test
# или коллекция в Postman
newman run api-tests.json
```

- ✓ Ключевые сценарии работают
- ✓ Нет критических ошибок
- ✓ Проверены роли и права доступа
- ✓ Результаты соответствуют ожиданиям



Post-deploy verification завершена

Приложение работает корректно, можно открывать доступ пользователям.



Проверяйте сразу после деплоя



Используйте метрики и алерты



Следите за логами в первые часы



Проверьте ключевые сценарии



6. Logs и наблюдение после release

Первые минуты после обновления особенно важны

Что смотреть после release

- Запустился ли service без crash loop.
- Нет ли новых 500 errors.
- Не появились ли Prisma connection errors.
- Не выросли ли 401/403 там, где ожидается 200.
- Не падают ли requests на новых routes.
- Проходит ли /health/ready стабильно.

journalctl: последние logs

BASH

```
sudo journalctl -u internet-shop -n 200 --no-pager
```

journalctl: live monitoring

BASH

```
sudo journalctl -u internet-shop -f
```

Nginx access/error logs

BASH

```
sudo tail -f /var/log/nginx/access.log  
sudo tail -f /var/log/nginx/error.log
```

Поиск причины по logs

Признак	Возможная причина	Где смотреть
Service exits immediately	Startup validation failed	journalctl
502 Bad Gateway	Node.js service не отвечает	systemctl, journalctl, nginx error.log
500 after migration	Код не совместим со схемой/данными	application logs
Prisma connection error	DATABASE_URL или PostgreSQL	journalctl, PostgreSQL status
CORS error	CORS_ORIGIN	browser console, app config
401 after update	JWT_SECRET или auth logic	application logs, request tests

Request ID после release

- Request ID помогает связать client error и server log.
- Если пользователь сообщает requestId, можно найти конкретный запрос.
- Логи должны показывать method, path, status, requestId и error summary.
- Не записывайте в logs passwords, tokens и полные secrets.

Наблюдение после release

Следим за здоровьем приложения, чтобы вовремя заметить и решить проблемы

- ✓ Раннее обнаружение проблем
- ✓ Быстрая реакция
- ✓ Минимизация простоев
- ✓ Довольные пользователи

Стабильная
работа —
лучший результат
релиза!

1

Логи

Проверяем ошибки
и важные события



```
2026-09-21 10:15:32 INFO
Server started on port 3000

2026-09-21 10:16:01 ERROR
Error: Cannot read property
'id' of undefined
    at getUser (routes/user.js:42)

2026-09-21 10:16:01 WARN
Slow query: 1250 ms /api/users
```

- ✓ Ошибки (ERROR)
- ✓ Предупреждения (WARN)
- ✓ Подозрительная активность
- ✓ Корреляция по request id

2

Метрики

Контролируем ключевые
показатели



HTTP Requests / sec	124
Response Time (p95)	230 ms
CPU Usage	32%
Memory Usage	1.2 GB

- ✓ RPS (запросов в секунду)
- ✓ Время ответа (p50/p95/p99)
- ✓ Использование CPU / RAM
- ✓ Активные соединения
- ✓ Ошибки (4xx / 5xx)

3

Трейсы

Анализируем цепочку
запроса



```
GET /api/orders/123 320 ms
├── nginx:1
│   ├── express 45 ms
│   │   ├── db.query 180 ms
│   │   └── redis 40 ms
│   └── external API 35 ms
```

- ✓ Где возникает задержка
- ✓ Время каждого сервиса
- ✓ Поиск узких мест
- ✓ Трассировка сквозного запроса

4

Алерты

Получаем уведомления
о проблемах



🔥 High error rate	10:18
5xx > 5% for 5 minutes	
⚠️ High response time	10:21
p95 > 1000 ms	
⚠️ High CPU usage	10:24
CPU > 80%	
ℹ️ Service down	10:25
node-api.service is not active	

- ✓ Настроенные пороги
- ✓ Уведомления (Telegram/Email)
- ✓ Эскалация при критичных сбоях
- ✓ Интеграция с мониторингом

5

Дашборды

Видим общую картину
в реальном времени



Production Dashboard 🟢 Healthy	
RPS 124	Response Time (p95) 230 ms
Error Rate 0.3%	CPU / Memory 32% / 1.2 GB

- ✓ Ключевые метрики на одном экране
- ✓ Статус всех сервисов
- ✓ История и тренды
- ✓ Быстрый переход к логам и трассам



Если обнаружена
проблема



1. Анализируем
Смотрим логи, метрики,
трейсы



2. Определяем причину
Находим корневую
проблему



3. Принимаем решение
Фикс, конфигурация
или откат



4. Применяем изменения
Деплой/конфиг/rollback



5. Проверяем результат
Убеждаемся, что система
стабильна



Наблюдение = стабильный продакшн

Чем раньше мы видим проблему, тем легче и дешевле её решить.



Используйте
метрики и алерты



Настройте
уведомления



Держите runbook
с действиями



Постоянно
улучшайте мониторинг

Следи.
Анализируй.
Улучшай!

7. Rollback: возврат предыдущей версии

Что можно откатить быстро, а что требует осторожности

Что такое rollback

- Rollback — управляемый возврат к предыдущей рабочей версии приложения или конфигурации после неудачного release.
- Rollback приложения обычно проще, чем rollback database schema.
- Нужно знать предыдущий рабочий commit.
- Rollback должен завершаться health checks и smoke tests.
- После rollback нужно зафиксировать причину incident.

Rollback кода и rollback базы

Код

- `git checkout` предыдущего commit/tag.
- `npm ci`.
- `prisma generate`.
- `systemctl restart`.
- Проверка health/smoke.

Database

- Может содержать необратимые изменения.
- Down migration не всегда безопасна.
- Backup restore может потерять новые данные.
- Нужен отдельный план для schema/data.

Rollback приложения на предыдущий commit

BASH

```
sudo -iu shop
cd /var/www/internet-shop

git log --oneline -10
git checkout <PREVIOUS_GOOD_COMMIT>
npm ci
npx prisma generate

exit
sudo systemctl restart internet-shop
```

Проверка после rollback

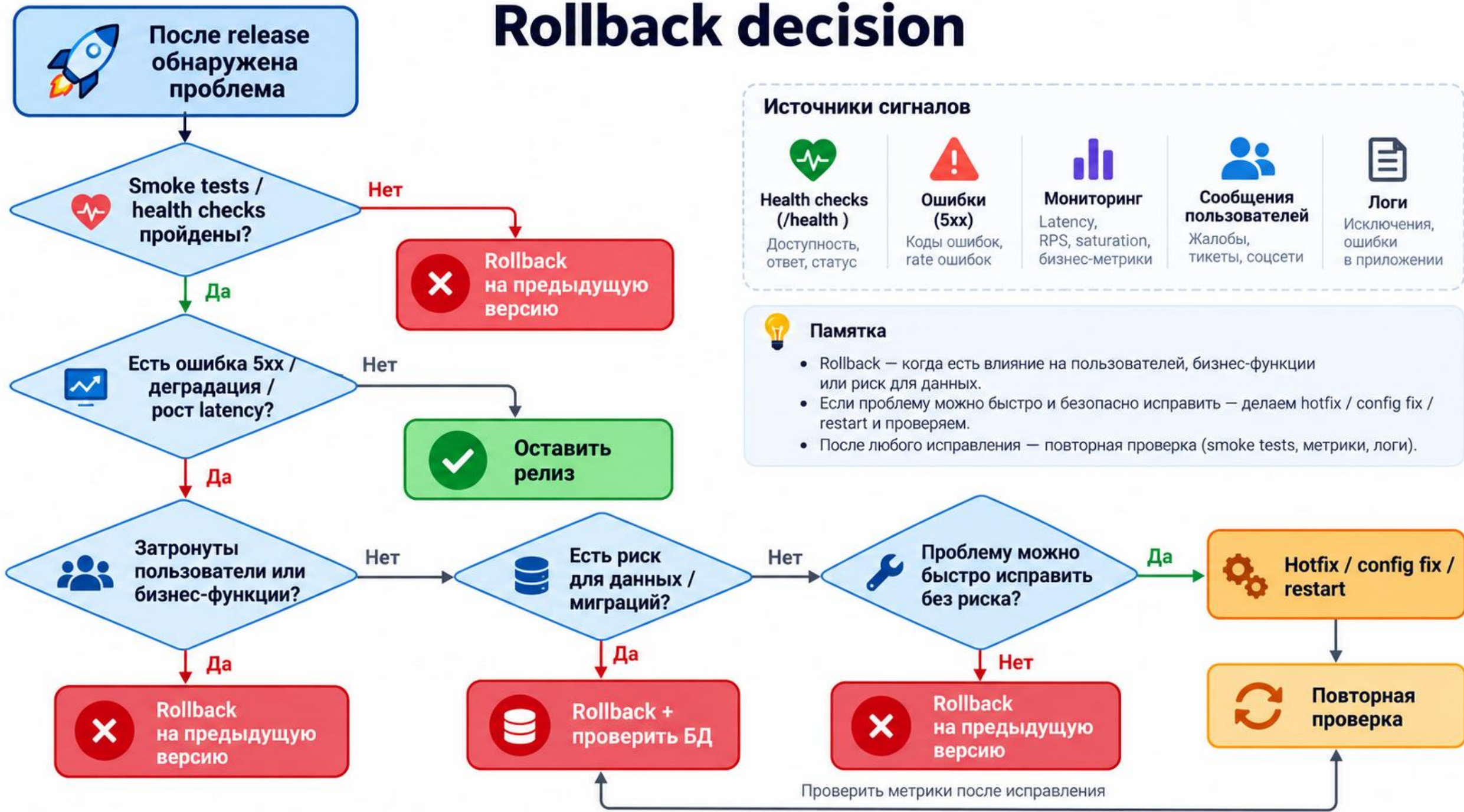
BASH

```
sudo systemctl status internet-shop --no-pager  
curl http://127.0.0.1:3000/health/ready  
curl http://SERVER_IP/api/products  
sudo journalctl -u internet-shop -n 100 --no-pager
```

Когда rollback приложения может не помочь

- Новая migration уже удалила column, нужную старому коду.
- Новая версия изменила формат данных.
- Новая версия изменила JWT_SECRET или auth policy.
- Новая версия создала несовместимые records.
- В этом случае rollback требует отдельной работы с database и данными.

Rollback decision





8. Обновление на Render

РaaS берёт часть операций на себя, но проверки всё равно нужны

Что меняется на Render

- Код приходит из GitHub repository.
- Render выполняет build command.
- Environment variables задаются в dashboard.
- Deploy logs доступны через Render UI.
- Service restart выполняется платформой.
- Health check и public URL доступны сразу после deployment.

Render update flow

- 1 Push** Отправить commit в deployment branch.
- 2 CI** Дождаться GitHub Actions PASS, если настроена проверка.
- 3 Deploy** Render запускает build/start workflow.
- 4 Migrations** В учебном варианте start command выполняет db:deploy.
- 5 Health Check** Render проверяет /health/ready.
- 6 Verify** Проверить API вручную и открыть logs.

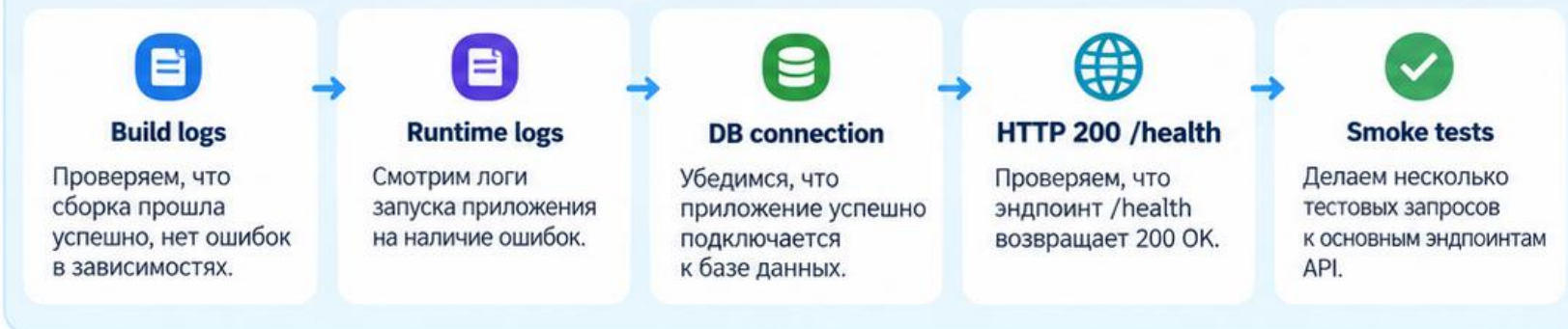
Render update

Схема обновления Node.js REST API на платформе Render



Диагностика:
logs / config / migrations
Если health check не прошёл, версия помечается как неудачная. Анализируем логи и конфигурацию.

Что проверяем после update



- Важно**
- 1 Секреты не хранятся в .env в репозитории (используем Environment Variables в Render).
 - 2 Миграции выполняются до полной готовности версии (чтобы избежать несоответствия схемы и кода).
 - 3 При ошибке обновления анализируют logs и health check (смотрим build logs, runtime logs, конфигурацию и миграции).

Rollback на Render

- В интерфейсе Render можно использовать deploy history.
- Можно redeploy-ить предыдущий successful deploy.
- Если изменялась database schema, применяются те же ограничения, что и на VPS.
- После rollback всё равно выполняются health checks и smoke tests.
- Нужно проверить, что environment variables соответствуют версии приложения.

Linux update vs Render update

Linux server

- Команды выполняет администратор.
- `systemd restart` вручную.
- Nginx и firewall под контролем администратора.
- Logs через `journalctl` и `/var/log/nginx`.
- Больше контроля и ответственности.

Render

- Build/deploy запускает платформа.
- Restart выполняет платформа.
- Public URL и HTTPS даёт платформа.
- Logs через dashboard.
- Меньше системного администрирования.



9. Регулярное сопровождение Production

Не только deploy: backups, updates, secrets, monitoring

Еженедельный/регулярный production checklist



Проверить health endpoints.



Просмотреть application error logs.



Проверить свободное место на диске.



Проверить status systemd services.



Проверить, выполняются ли backups.



Проверить доступность PostgreSQL.



Проверить security updates ОС.



Проверить, не истекает ли TLS certificate.



Проверить dependencies на known vulnerabilities.



Актуализировать release notes.

Backups должны быть проверяемыми

- Backup без проверки восстановления может оказаться бесполезным.
- Нужно знать, где хранится backup.
- Нужно знать, кто имеет к нему доступ.
- Нужно периодически выполнять test restore в отдельной среде.
- В отчёт не помещают реальные production dumps.

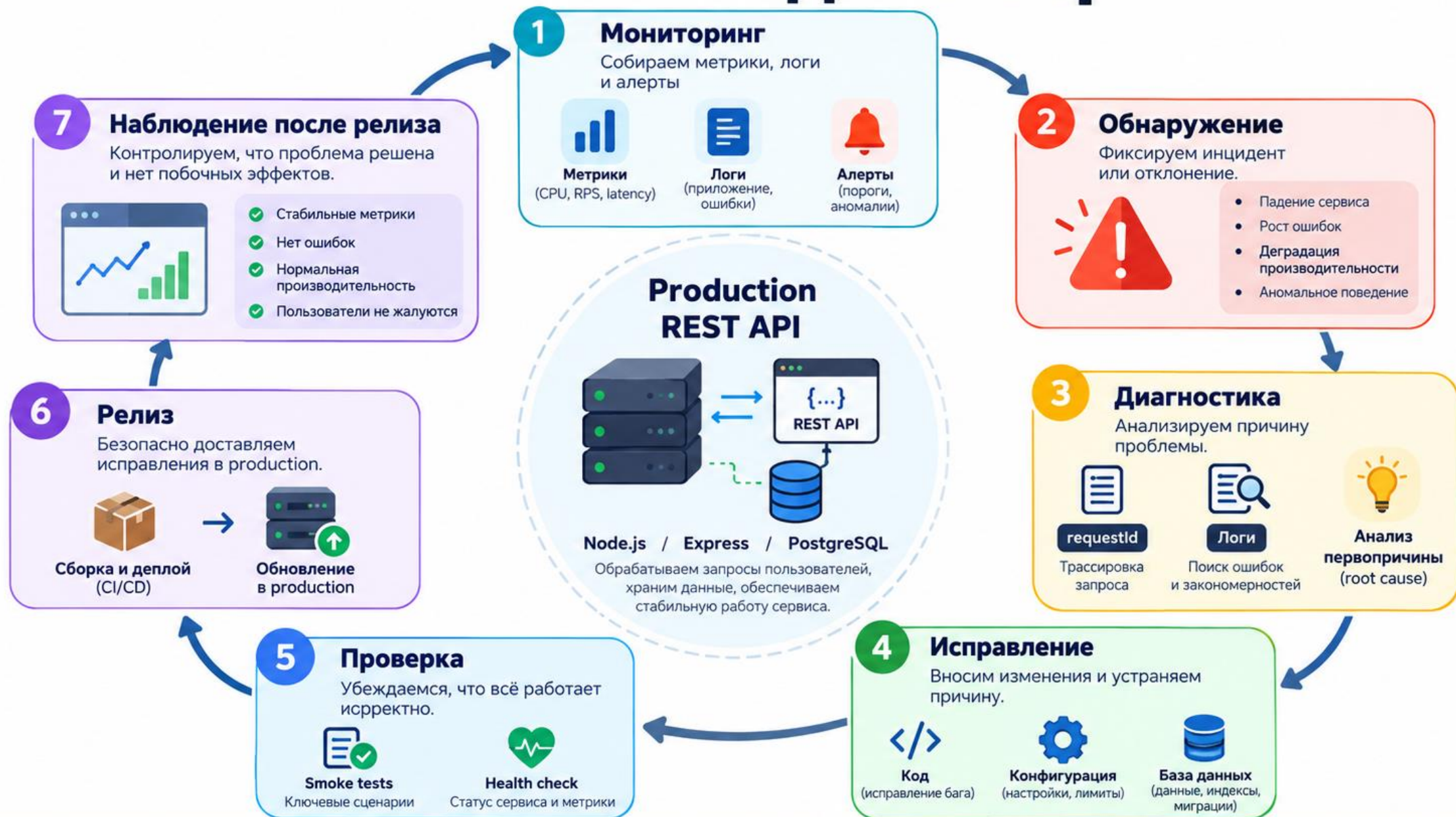
Security updates

- Операционная система должна получать security updates.
- Node.js LTS нужно обновлять в пределах поддерживаемой ветки после тестирования.
- npm dependencies нужно обновлять осознанно и через CI.
- Secrets нужно менять при подозрении на утечку.
- Неиспользуемые accounts и keys нужно удалять.

Минимальный incident report

- Когда началась проблема.
- Какой release был активен.
- Какие endpoints пострадали.
- Какие logs и requestId подтверждают проблему.
- Что было сделано для исправления.
- Был ли rollback.
- Как предотвратить повторение.

Production support loop



Поддержка production — это непрерывный цикл, а не разовое действие.

Практическая задача

Выполнить учебное обновление Internet Shop REST API в production-подобной среде и подтвердить его работоспособность.

Задание

- Добавить небольшой version marker или безопасное изменение endpoint.
- Зафиксировать commit.
- Проверить CI или локальные tests.
- Сделать backup PostgreSQL.
- Обновить Linux deployment или Render service.
- Выполнить migrations, если они есть.
- Перезапустить/redeploy сервис.
- Проверить health endpoints и smoke tests.
- Просмотреть logs.
- Смоделировать rollback на предыдущий commit или deploy.

Критерии проверки

- Новая версия доступна по public API.
- Health checks проходят.
- Smoke tests проходят.
- Logs не показывают новых критических ошибок.
- Есть backup перед изменением database.
- Показан понятный rollback plan.

Проверка понимания

- 1 Чем release отличается от deployment?
- 2 Почему перед database migration нужен backup?
- 3 Почему rollback кода проще rollback database schema?
- 4 Какие команды нужны для обновления Linux deployment?
- 5 Что проверяет smoke test?
- 6 Где искать ошибки после systemctl restart?
- 7 Какой минимальный набор production logs нужен для диагностики?
- 8 Чем update на Render отличается от update на VPS?
- 9 Почему нельзя хранить production secrets в repository?

Итоги лекции

1

Production release — это не только запуск новой версии, но и подтверждение её работоспособности.

2

Перед обновлением нужны CI status, понимание changes, backup и rollback plan.

3

Linux update обычно включает git pull, npm ci, prisma generate, migrate deploy и systemctl restart.

4

Database migrations требуют особой осторожности и backup.

5

После deployment выполняются health checks, smoke tests и анализ logs.

6

Rollback приложения возможен через предыдущий commit/tag, но database rollback сложнее.

7

На Render часть инфраструктурных операций выполняет платформа, но проверки и logs остаются обязательными.

8

Production сопровождение включает backups, updates, secrets, monitoring и incident reports.

9

PO 4.5 завершает полный backend-cycle: тестировать → диагностировать → подготовить → развернуть → сопровождать.